پاسخ‌های تمرین اول طراحی پایگاه داده‌ها

نیم‌سال اول ۱۴۰۵ — تهیه‌شده بر پایه‌ی اسلایدهای «بخش اول: مقدمه‌ای بر سیستم‌های پایگاهی»

سؤال ۱ — پاسخ کوتاه

۱. دو ضعف سامانه پایگاه داده نسبت به سامانه فایل

- **هزینه و پیچیدگی بیشتر:** DBMS نرم‌افزاری سنگین و گران است و به سخت‌افزار قوی‌تر و مدیر پایگاه (DBA) نیاز دارد. مثال: برای نگهداری فهرست ساده‌ی مخاطبین یک برنامه، راه‌اندازی یک DBMS به‌صرفه نیست.
- **سربار کارایی و تمرکز ریسک:** کنترل همروندی، تراکنش، امنیت و ثبت log سرعت را کم می‌کند و خرابی DBMS می‌تواند همه‌ی برنامه‌ها را از کار بیندازد. مثال: پردازش یک‌باره‌ی یک فایل متنی بزرگ با یک برنامه‌ی ساده معمولاً از اجرای همان کار از طریق DBMS سریع‌تر است.

۲. سیستم واقعی که مشی فایلی در آن قابل توجیه است

ذخیره‌ی فایل‌های log یک وب‌سرور: داده‌ها فقط به انتهای فایل افزوده می‌شوند، ساختار ساده و ترتیبی دارند، پرس‌وجوی پیچیده یا ارتباط بین داده‌ها لازم نیست و کاربر همزمان زیادی روی آن‌ها تغییر نمی‌دهد. مثال‌های دیگر: فایل‌های تنظیمات (config) یک برنامه و ذخیره‌ی عکس و ویدیو.

۳. مثال واقعی برای انواع داده از نظر ساختار

- **ساخت‌یافته:** جدول دانشجویان با ستون‌های شماره دانشجویی، نام، رشته (فرمت ثابت و از پیش تعیین‌شده).
- **نیمه‌ساخت‌یافته:** فایل JSON یا XML (خودتوصیف‌گر)، مثلاً پاسخ یک API وب یا یک ایمیل با سرایند و متن.
- **بدون ساختار:** ویدیوی ضبط‌شده‌ی کلاس، تصویر، فایل صوتی یا متن آزاد یک مقاله.

۴. افزونگی کنترل‌شده و کنترل‌نشده

- **کنترل‌نشده (نامطلوب):** تکرار داده‌ها به‌دلیل استفاده از مشی غیرپایگاهی، تحت کنترل سیستم‌های مختلف که از هم خبر ندارند. مثال: اطلاعات یک دانشجو هم در فایل اداره‌ی آموزش و هم در فایل امور دانشجویی ذخیره شده و تغییر یکی در دیگری اعمال نمی‌شود.
- **کنترل‌شده (مجاز):** تکرار آگاهانه که DBMS از آن خبر دارد و آن را هماهنگ نگه می‌دارد. دو نوع دارد: طبیعی (مثلاً رشته‌ی یکسان برای چند دانشجو) و تکنیکی (مثلاً نمایه‌سازی یا کپی پشتیبان).

۵. نمایه‌سازی و افزونگی تکنیکی

نمایه (مثلاً درخت B یا +B) مقدار فیلد کلید جستجو، مانند شماره دانشجویی، را به‌همراه اشاره‌گر به رکورد در ساختار جداگانه‌ای نگه می‌دارد؛ پس مقدار کلید هم در داده و هم در نمایه تکرار می‌شود و این یک افزونگی تکنیکی و کنترل‌شده است. با این حال مفید است، چون جستجو به‌جای پیمایش خطی همه‌ی رکوردها، با چند دسترسی به دیسک در درخت چندسطحی انجام می‌شود. هزینه‌اش فضای بیشتر و کندتر شدن درج، حذف و به‌روزرسانی است، چون نمایه هم باید به‌روز شود.

۶. تفاوت طراحی معنایی، منطقی و فیزیکی

- **معنایی (مدل‌سازی معنایی):** مستقل از DBMS و پیاده‌سازی؛ شناسایی موجودیت‌ها، ویژگی‌ها، روابط و محدودیت‌های دنیای واقعی (مثلاً مدل ER).
- **منطقی:** تبدیل مدل معنایی به مدل داده‌ی DBMS هدف (مثلاً جدول‌ها، کلیدها و نرمال‌سازی در مدل رابطه‌ای) بدون پرداختن به جزئیات ذخیره‌سازی.
- **فیزیکی:** نحوه‌ی ذخیره روی دیسک: ساختار فایل‌ها، نمایه‌ها، بلوک‌بندی و تنظیمات کارایی.

۷. آیا طراحی UI و برنامه‌های کاربردی بخشی از پروژه‌ی پایگاه داده است؟

بله. در مشی پایگاهی، «پروژه‌ی پایگاهی» علاوه بر طراحی معنایی، منطقی و فیزیکی، شامل طراحی UI، طراحی برنامه‌های کاربردی (همراه با تعریف تراکنش‌ها) و تولید برنامه‌های تعریف، کنترل و پردازش داده‌ها نیز هست. این بخش‌ها لایه‌ی بالایی پروژه‌اند و بر پایه‌ی طرح پایگاه ساخته می‌شوند، نه هسته‌ی خود پایگاه.

۸. تفاوت مدل داده و پیاده‌سازی آن، و اهمیت آن

- **مدل داده:** مجموعه‌ای انتزاعی از مفاهیم (ساختار، روابط، محدودیت‌ها و عملیات) برای توصیف داده؛ مثلاً مدل رابطه‌ای که در سال ۱۹۷۰ به‌صورت نظریه مطرح شد.
- **پیاده‌سازی:** یک DBMS مشخص (مثلاً MySQL یا Oracle) که آن مدل را با ساختارهای ذخیره‌سازی و الگوریتم‌های معین عملی می‌کند (پیاده‌سازی مدل رابطه‌ای در دهه‌ی ۱۹۸۰).
- **اهمیت:** این جداسازی استقلال برنامه‌ها از داده‌ها (انتزاعی‌سازی) را ممکن می‌کند. می‌توان پیاده‌سازی را بدون تغییر مدل و برنامه‌ها بهینه یا عوض کرد، طراح بر مفاهیم تمرکز می‌کند و محصولات مختلف را می‌توان با یک مدل مشترک مقایسه کرد.

۹. کدام مورد از اجزای مدل داده نیست؟

Data Sequence. سه مورد دیگر، یعنی Data Semantic (معنای داده)، Data Relationship (روابط) و Data Constraint (محدودیت‌ها)، از اجزای مدل داده‌اند.

۱۰. کدام مؤلفه از ثبت‌نام دو نفر در درسی با یک ظرفیت خالی جلوگیری می‌کند؟

مدیریت تراکنش و کنترل همروندی (Concurrency Control) در DBMS. با قفل‌گذاری، دو تراکنش ثبت‌نام همزمان به‌صورت منزوی (ویژگی Isolation) اجرا می‌شوند؛ اولی ظرفیت را پر می‌کند و دومی ظرفیت پر را می‌بیند و رد می‌شود. محدودیت جامعیت «تعداد ثبت‌نام ≥ ظرفیت» نیز در این کار نقش دارد.

۱۱. برداشت از A1 و خطا پیش از انتقال به A2

ویژگی **اتمیک بودن (Atomicity)**: تراکنش یا باید کامل اجرا شود یا هیچ‌کدام از اثرهایش باقی نماند. با بروز خطا، برداشت از A1 لغو می‌شود (rollback) تا مجموع A1+A2 ثابت بماند. این موضوع به سازگاری (Consistency) هم مربوط است، اما علت مستقیم و ویژگی اصلی همان Atomicity است.

۱۲. تأثیر نرخ گرفتن snapshot (عکس فوری)

هرچه snapshotها پرتکرارتر باشند:

- **کارایی حین عملکرد عادی:** کاهش می‌یابد، چون هر snapshot سربار I/O، پردازش و فضای ذخیره‌سازی دارد.

- **سرعت بازیابی پس از خرابی:** افزایش می‌یابد، چون آخرین snapshot به لحظه‌ی خرابی نزدیک‌تر است و تغییرات کمتری باید بازپخش شود.
- **اطلاعات ازدست‌رفته:** کاهش می‌یابد، چون فاصله‌ی بین آخرین snapshot و لحظه‌ی خرابی کوتاه‌تر است.

نرخ کم عکس‌گیری اثر برعکس دارد: عملکرد عادی بهتر، اما بازیابی کندتر و ازدست‌رفتن اطلاعات بیشتر.

۱۳. چرا دیسک رسانه‌ی اصلی و نوار مغناطیسی رسانه‌ی فرعی است؟

دیسک دسترسی مستقیم (تصادفی) دارد و هر رکورد را در چند میلی‌ثانیه برمی‌گرداند، پس برای پرس‌وجو و تراکنش‌های برخط مناسب است. نوار فقط دسترسی ترتیبی دارد و رسیدن به یک رکورد به پیمایش طولانی نیاز دارد. در عوض، نوار هزینه‌ی بسیار کمتری به ازای هر بیت (CPB) دارد، ظرفیت و ماندگاری بالا و قابلیت جابه‌جایی دارد؛ بنابراین برای نسخه‌های پشتیبان و بایگانی (رسانه‌ی فرعی) مناسب است.

۱۴. اثر بازسازی دیسک جایگزین بر کارایی: Disk Mirroring در برابر Striping+Parity

- **Disk Mirroring (مثل RAID 1 و RAID 10):** بازسازی فقط کپی کردن بلوک‌ها از دیسک آینه‌ی سالم به دیسک جدید است. فقط یک دیسک خوانده می‌شود، محاسبه‌ای لازم نیست، بار روی بقیه‌ی دیسک‌ها کم است و بازسازی سریع انجام می‌شود؛ افت کارایی سامانه‌ی در حال اجرا کم است.
- **Striping+Parity (مثل RAID 5 و RAID 6):** برای بازسازی هر بلوک باید همه‌ی دیسک‌های باقی‌مانده خوانده و داده با محاسبه‌ی parity (مثلاً XOR) بازسازی شود. بار I/O و CPU زیاد است، فرایند طولانی است و کارایی سامانه چشمگیر افت می‌کند. در این مدت هر خواندن از دیسک خراب نیز به محاسبه نیاز دارد. همچنین احتمال خرابی دیسک دوم در این بازه بیشتر است که در RAID 5 به از دست رفتن داده می‌انجامد.

۱۵. معماری متمرکز برای چه کاربردهایی مناسب‌تر است؟

برای کاربردهایی که داده و کاربران عمدتاً در یک مکان‌اند، حجم داده و بار متوسط است و کنترل و امنیت یکپارچه اهمیت دارد؛ مثلاً سامانه‌ی حسابداری و انبار یک شرکت، سامانه‌ی آموزش یک دانشکده یا امانت کتابخانه‌ی یک مرکز. مدیریت و حفظ سازگاری آسان است، ولی نقطه‌ی شکست واحد و تنگنای کارایی دارد.

۱۶. دلیل اصلی معماری ۳ لایه برای برنامه‌های تحت وب

جداسازی لایه‌ی ارائه (مرورگر و UI)، لایه‌ی منطق کاربردی (وب‌سرور و برنامه) و لایه‌ی داده (DBMS). نتیجه: هر لایه مستقل تغییر و مقیاس پیدا می‌کند، نگهداری ساده‌تر می‌شود، کاربر مستقیماً به پایگاه دسترسی ندارد (امنیت بیشتر)، اتصال‌های پایگاه به‌صورت اشتراکی مدیریت می‌شوند و روی کلاینت چیزی جز مرورگر نصب نمی‌شود.

۱۷. پایگاه داده‌ی منطقاً مجتمع ولی فیزیکی توزیع‌شده

کاربر و برنامه‌ها یک طرح و یک دید یکپارچه (یک schema سراسری) می‌بینند، ولی داده‌ها روی چند سرور یا سایت پخش شده‌اند (با قطعه‌بندی یا تکرار) و DBMS توزیع‌شده با «شفافیت توزیع» محل واقعی داده‌ها را پنهان می‌کند. مثال: پایگاه دانشگاهی با سه سایت تهران، مشهد و شیراز که کاربر آن را یک جدول «دانشجو» می‌بیند، ولی رکوردها روی سه سرور ذخیره شده‌اند.

سؤال ۲ — افزونگی و ناسازگاری در دو فایل دانشجویی

وقتی اداره‌ی آموزش و امور دانشجویی هر کدام فایل جداگانه‌ای با فیلدهای شماره دانشجویی، نام، رشته، نشانی و تلفن دارند، هر دانشجو دو بار ذخیره می‌شود و دو نسخه بدون هیچ هماهنگی خودکاری از هم جدا می‌شوند.

سناریو	چه اتفاقی می‌افتد	نوع مشکل
ذخیره‌ی تکراری	همه‌ی فیلدهای هر دانشجو در دو فایل نگهداری می‌شود؛ فضا دو برابر مصرف می‌شود و هر تغییر باید دو بار و توسط دو نفر ثبت شود.	افزونگی نامطلوب
تغییر نشانی یا تلفن	دانشجو نشانی را فقط به امور دانشجویی اعلام می‌کند؛ آموزش نشانی قدیمی را نگه می‌دارد و نامه‌ها و کارنامه به آدرس اشتباه می‌رود.	ناسازگاری در به‌روزرسانی
تغییر رشته	رشته در آموزش اصلاح می‌شود ولی در امور دانشجویی قدیمی می‌ماند؛ خدمات رفاهی و آمار بر پایه‌ی رشته‌ی نادرست محاسبه می‌شود.	ناسازگاری در به‌روزرسانی
ثبت یا حذف ناهماهنگ	دانشجوی جدید فقط در یک فایل ثبت می‌شود، یا دانشجوی منصرف فقط از یک فایل حذف می‌شود؛ آن دانشجو در یکی از دو اداره «هست» و در دیگری «نیست».	ناسازگاری در درج و حذف
قالب‌های متفاوت	یک نام با دو املا («علیرضا» و «علی‌رضا») یا تلفن با دو قالب نوشته می‌شود؛ تطبیق رکوردها و گزارش مشترک دو اداره غیرممکن یا خطادار می‌شود.	ناسازگاری در قالب داده

معماری پایگاهی چگونه این مشکل‌ها را کم می‌کند؟

- **یکپارچگی داده:** هر دانشجو در یک جدول مشترک (مثلاً §§P1§§ با کلید اصلی StdNo) فقط یک‌بار ذخیره می‌شود. داده‌های خاص هر اداره در جدول‌های جدا و با ارجاع به StdNo نگهداری می‌شوند. پس افزونگی کنترل‌نشده حذف می‌شود.
- **یک‌جا به‌روز کردن:** تغییر نشانی یا رشته یک‌بار انجام می‌شود و هر دو اداره بلافاصله مقدار جدید را می‌بینند.
- **محدودیت‌های جامعیت:** کلید اصلی از تکرار جلوگیری می‌کند و قیدهای دامنه و قالب (مثلاً قالب تلفن) از ورود مقادیر ناسازگار جلوگیری می‌کنند.
- **تراکنش و کنترل همروندی:** به‌روزرسانی همزمان دو اداره اتمیک و منزوی اجرا می‌شود و ناسازگاری میانی پیش نمی‌آید.
- **دیدهای مختلف (View):** هر اداره بدون داشتن نسخه‌ی جدا، فقط ستون‌ها و ردیف‌های مربوط به خود را می‌بیند (با کنترل دسترسی).
- **افزونگی کنترل‌شده:** تکرار لازم (نمایه، کپی پشتیبان) را خود DBMS هماهنگ و سازگار نگه می‌دارد.

سؤال ۳ — مقایسه‌ی سطوح RAID با ۸ دیسک هم‌اندازه

فرض: ظرفیت هر دیسک S است، پس ظرفیت خام ۸S است. RAID 1 با ۸ دیسک یعنی یک آینه‌ی ۸ نسخه‌ای؛ اگر ۴ جفت آینه‌ای مدنظر باشد، همان RAID 10 است.

سطح	ظرفیت قابل استفاده	تحمل خرابی	کارایی خواندن	کارایی نوشتن	پیچیدگی بازسازی
RAID 0	۸S (۱۰۰٪)	هیچ؛ خرابی یک دیسک کل آرایه را از بین می‌برد	عالی (موازی روی ۸ دیسک)	عالی	بازسازی ندارد؛ فقط بازیابی از پشتیبان
RAID 1	S ≈ (۱۲٫۵٪)	تا ۷ دیسک	بالا (خواندن از هر نسخه)	معادل یک دیسک (هر نوشتن روی همه‌ی نسخه‌ها)	ساده و سریع (کپی از نسخه‌ی سالم)
RAID 5	۷S (۸۷٫۵٪)	۱ دیسک	خوب	ضعیف‌تر (هر نوشتن کوچک ≈ ۴ عملیات I/O به‌خاطر parity)	پیچیده و طولانی (خواندن همه‌ی ۷ دیسک باقی‌مانده)
RAID 6	۶S (۷۵٪)	۲ دیسک همزمان	خوب	ضعیف‌تر از RAID 5 (دو parity)	پیچیده‌تر و طولانی‌تر (محاسبه‌ی دو parity)
RAID 10	۴S (۵۰٪)	حداقل ۱ دیسک؛ تا ۴ دیسک اگر از هر جفت فقط یکی خراب شود (وابسته به الگوی خرابی)	عالی	خوب (دو نوشتن، بدون محاسبه‌ی parity)	ساده و سریع (کپی از آینه‌ی همان جفت)

انتخاب سطح مناسب برای سه کاربرد

کاربرد	سطح پیشنهادی	دلیل
انبار داده (خواندن زیاد، حجم زیاد، بارگذاری دسته‌ای)	RAID 6 (RAID 5 اگر هزینه مهم‌تر و پشتیبان منظم موجود باشد)	بیشترین ظرفیت قابل استفاده بعد از RAID 0 و خواندن خوب؛ جریمه‌ی نوشتن در بارگذاری دسته‌ای مهم نیست. با دیسک‌های پرظرفیت، بازسازی طولانی است و تحمل دو خرابی ریسک دیسک دوم را پوشش می‌دهد.
سامانه‌ی مالی حساس	RAID 10 (در صورت لزوم تضمین تحمل هر ۲ خرابی: RAID 6)	اولویت با تحمل خرابی و بازسازی سریع و ساده است تا پنجره‌ی خطر کوتاه شود و نوشتن بدون جریمه‌ی parity انجام شود؛ هزینه‌اش ۵۰٪ ظرفیت است. RAID جایگزین پشتیبان‌گیری نیست و پشتیبان هم لازم است.
سامانه‌ی تراکنشی پرترافیک (OLTP)	RAID 10	نوشتن و خواندن تصادفی کوچک و زیاد؛ RAID 5 و RAID 6 در این الگو به‌خاطر جریمه‌ی نوشتن parity کند می‌شوند، ولی RAID 10 کارایی بالا و تأخیر کم دارد.

در عمل، هر دو سامانه‌ی مالی و OLTP به RAID 10 می‌رسند، اما با دلیل متفاوت: تحمل خرابی و بازسازی ساده برای سامانه‌ی مالی، و برای OLTP، کارایی نوشتن.

سؤال ۴ — توزیع داده‌ها در پایگاه سه‌سایتی (تهران، مشهد، شیراز)

قاعده‌ی کلی: داده‌ای که بیشتر محلی استفاده می‌شود قطعه‌بندی، داده‌ی کوچک و پرخوان و کم‌تغییر تکرار، و داده‌ی حساس یا نیازمند کنترل یکپارچه متمرکز می‌شود.

داده	تصمیم	دسترس‌پذیری و کارایی	شفافیت توزیع	خطر ناسازگاری
ثبت‌نام دانشجویان	قطعه‌بندی افقی بر پایه‌ی سایت محل تحصیل (به‌همراه یک کپی پشتیبان از هر قطعه در یک سایت دیگر)	بیشتر دسترسی‌ها محلی و سریع است؛ با خرابی یک سایت، کپی پشتیبان در سایت دیگر در دسترس می‌ماند.	باید شفافیت قطعه‌بندی داشته باشد: کاربر یک جدول «ثبت‌نام» را می‌بیند و DBMS محل واقعی رکورد را پیدا می‌کند.	کم؛ هر رکورد یک سایت مالک دارد. فقط پرس‌وجوهای بین‌سایتی (مثلاً درس بین‌پردیسی) به تراکنش توزیع‌شده نیاز دارند.
اطلاعات پایه‌ی درس‌ها	تکرار کامل در هر سه سایت	بالاترین دسترس‌پذیری و سرعت خواندن: همه‌ی سایت‌ها آن را محلی می‌خوانند و با خرابی یک سایت مشکلی پیش نمی‌آید.	شفافیت تکرار: کاربر نمی‌داند کدام کپی را می‌خواند.	کم تا متوسط؛ داده‌ها کوچک و به‌ندرت تغییر می‌کنند، پس به‌روزرسانی از یک سایت مرجع به بقیه منتشر می‌شود.
گزارش فعالیت‌های مالی	متمرکز (مثلاً در تهران) با نسخه‌ی پشتیبان در یک سایت دیگر	کارایی دسترسی از مشهد و شیراز کمتر است و به دسترس‌پذیری تهران و شبکه وابسته است؛ نسخه‌ی پشتیبان این ریسک را کاهش می‌دهد.	شفافیت مکان: کاربران شهرهای دیگر بدون دانستن محل ذخیره از طریق یک واسط گزارش می‌گیرند.	بسیار کم؛ یک نسخه‌ی معتبر وجود دارد. کنترل امنیت، حسابرسی و تجمیع گزارش‌ها هم ساده می‌شود.
محتویات ارائه‌ی درس‌ها (اسلاید، ویدیو)	قطعه‌بندی بر پایه‌ی درس‌های ارائه‌شده در هر سایت، با تکرار گزینشی برای درس‌های مشترک یا پرمراجعه	هر سایت محتوای درس‌های خودش را سریع و محلی می‌دهد و حجم زیاد داده روی شبکه‌ی بین‌سایتی حرکت نمی‌کند؛ نسخه‌ی تکراری درس‌های مشترک، دسترس‌پذیری آن‌ها را بالا می‌برد.	شفافیت توزیع کامل: دانشجو با یک نشانی به محتوا می‌رسد.	کم؛ به‌روزرسانی معمولاً توسط استاد همان درس در یک سایت انجام می‌شود، و فقط کپی‌های گزینشی باید همگام شوند.

جمع‌بندی ملاحظات

- **تکرار:** دسترس‌پذیری و سرعت خواندن را بالا می‌برد، ولی خطر ناسازگاری و هزینه‌ی همگام‌سازی و فضا را زیاد می‌کند؛ پس فقط برای داده‌ی کوچک و کم‌تغییر مناسب است.
- **تمرکز:** ناسازگاری و پیچیدگی را حذف می‌کند، ولی دسترس‌پذیری و کارایی کاربران دورتر را کم می‌کند؛ مناسب داده‌ی حساس و تجمیعی.
- **قطعه‌بندی:** محلی بودن دسترسی و مقیاس‌پذیری را بالا می‌برد، ولی پرس‌وجوهای بین‌سایتی را گران و پیچیده می‌کند؛ مناسب داده‌ی حجیم با الگوی دسترسی محلی.